

BÀI GIẢNG

VIBE CODING

TẠO CODE PYTHON GIẢI QUYẾT CÁC BÀI TOÁN MẠCH ĐIỆN

Code với cảm hứng – Hiểu bằng trực quan – Giải quyết bằng Python

VIBE CODING LÀ GÌ?

- ✓ Kết hợp cảm hứng (Vibe) + lập trình (Coding)
- ✓ Tư duy trực quan – viết code tự nhiên
- ✓ Python là công cụ mạnh mẽ để phân tích và giải các bài toán mạch điện
- ✓ Nhanh – Linh hoạt – Dễ mở rộng

NỘI DUNG BÀI GIẢNG

1. Giới thiệu Vibe Coding & Python trong mạch điện
2. Công cụ & Thư viện Python hữu ích
3. Phân tích mạch DC cơ bản
4. Phân tích mạch AC (Phasor)
5. Đáp ứng quá độ mạch RLC
6. Bài toán nâng cao & Tối ưu
7. Thực hành dự án mini

QUY TRÌNH VIBE CODING GIẢI BÀI TOÁN MẠCH ĐIỆN

Hiểu bài toán
Xác định yêu cầu, tham số, điều kiện

Mô hình mạch
Vẽ sơ đồ mạch & chọn phương pháp

$f(x)$
 $f(x)$

Lập mô hình toán học
 $Ax = b$, hàm truyền, phương trình vi phân...

Vibe Coding với Python
Viết code, kiểm tra, thêm trực quan hóa

Phân tích & Kết luận
Đánh giá kết quả, rút ra nhận xét

Prof Happy AI

VÍ DỤ MINH HỌA

1. MẠCH DC – NODAL ANALYSIS

Sơ đồ mạch

Python Code

```
import numpy as np
# Ma trận dẫn nạp G (Siemens)
G = np.array([[1/1000+1/2000+1/3000, -1/2000],
              [-1/2000, 1/2000]])
# Vector nguồn dòng I (Ampe) đi vào các nút
I = np.array([10, 0])
# Giải hệ phương trình
Y = np.linalg.solve(G, I)
print("Y = ", Y)
# Kết quả
V1 = 6.000 V, V2 = 4.000 V
```

2. MẠCH AC – PHASOR

Sơ đồ mạch

Python Code

```
import numpy as np
w = 314.159 # rad/s
R = 100
L = 0.1
C = 10e-6
Z = R + 1j*w*L - 1j/(w*C)
V1 = 10 # V
I = V1 / Z
print("I = ", I)
# Kết quả
I = 0.615 A ∠ -36.87°
```

3. ĐÁP ỨNG QUÁ ĐỘ MẠCH RLC

Mạch RLC nối tiếp – Kích DC

Python Code

```
import numpy as np
from scipy.integrate import odeint
R, L, C, V0 = 100, 0.1, 10e-6, 10
def fcn(y, t):
    i, vC = y
    di_dt = (V0 - vC - R*i) / L
    dvC_dt = i / C
    return [di_dt, dvC_dt]
t = np.linspace(0, 0.05, 1000)
y0 = [0, 0]
sol = odeint(fcn, y0, t)
i, vC = sol[:, 0], sol[:, 1]
plt.plot(t, i, label='Current i(t)')
plt.plot(t, vC, label='Capacitor voltage vC(t)')
plt.legend()
plt.show()
```

Kết quả

4. MẠCH PHỨC TẠP – PHÂN TÍCH MẠNG

Sơ đồ mạch (ví dụ)

Python (NetworkX)

```
import networkx as nx
G = nx.DiGraph()
# Thêm các nút và cạnh có trọng số
G.add_edge(1, 2, weight=R1)
G.add_edge(2, 3, weight=R2)
G.add_edge(3, 4, weight=R3)
G.add_edge(4, 5, weight=R4)
G.add_edge(5, 6, weight=R5)
# Phân tích mạng, tính điện áp, dòng...
# (Áp dụng các định luật Kirchhoff)


Kết quả



Điện áp nút, dòng nhánh, công suất...


```

ƯU ĐIỂM

- Trực quan
- Nhanh chóng
- Linh hoạt
- Dễ mở rộng
- Chính xác

ỨNG DỤNG THỰC TIỄN

- Thiết kế & kiểm tra mạch điện
- Phân tích hệ thống điện – điện tử
- Nghiên cứu & học tập
- Tự động hóa & tối ưu hóa

THÔNG DIỆP

Vibe Coding giúp bạn biến ý tưởng thành code, biến code thành hiểu biết, và biến hiểu biết thành giải pháp thực tiễn!

□ Bài giảng về Vibe Coding tạo code để giải quyết các bài toán mạch điện bằng Python,

Sưu tầm và Biên soạn bổ sung phần Vibe Coding

With the support of Gemini

Prof Happy AI

Date: May 9, 2026

Double-click (or enter) to edit

Để giải quyết các bài toán mạch điện bằng Python, chúng ta thường sử dụng thư viện NumPy để xử lý ma trận và SciPy nếu cần giải các hệ phương trình vi phân phức tạp.

Dưới đây là 3 dạng bài toán điển hình kèm mã nguồn Python chi tiết:

□ 1. Giải mạch bằng phương pháp Điện thế nút (Nodal Analysis)

Đây là bài toán cơ bản nhất: Giải hệ phương trình $G \cdot V = I$ để tìm điện thế tại các nút. Bài toán: Cho mạch điện có 3 nút (không tính nút gốc). Các giá trị điện trở giữa các nút và nguồn dòng tại các nút đã biết.

```
import numpy as np

# Ma trận dẫn nạp G (Siemens - S = 1/Ohm)
# Giả sử các giá trị điện trở giữa các nút đã được tính toán
G = np.array([
    [ 0.7, -0.2, -0.5],
    [-0.2,  1.2, -1.0],
    [-0.5, -1.0,  1.5]
])

# Vector nguồn dòng I (Ampe) đi vào các nút
I = np.array([2, 0, -1])
```

```

try:
    # Giải hệ phffơng trình  $G * V = I$  bằng hàm linalg.solve-->Math!!!!!!
    V = np.linalg.solve(G, I)

    print("Điện thế tại các nút (V):")
    for i, v in enumerate(V):
        print(f"Nút {i+1}: {v:.2f} V")
except np.linalg.LinAlgError:
    print("Ma trận G suy biến, không thể giải mạch.")

```

Ma trận G suy biến, không thể giải mạch.

```

import numpy as np

# Ma trận dẫn nạp G
# Tăng G[2,2] lên một chút (ví dụ thêm 0.1S nối đất tại nút 3)
G = np.array([
    [ 0.7, -0.2, -0.5],
    [-0.2,  1.2, -1.0],
    [-0.5, -1.0,  1.6] # Thay 1.5 thành 1.6
])

I = np.array([2, 0, -1])

try:
    V = np.linalg.solve(G, I)
    print("Điện thế tại các nút (V):")
    for i, v in enumerate(V):
        print(f"Nút {i+1}: {v:.2f} V")
except np.linalg.LinAlgError:
    print("Ma trận G vẫn suy biến!")

```

Điện thế tại các nút (V):
 Nút 1: 13.00 V
 Nút 2: 10.50 V
 Nút 3: 10.00 V

Để biểu diễn trực quan kết quả này, chúng ta có thể sử dụng thư viện Matplotlib. Có hai cách tiếp cận hiệu quả: vẽ biểu đồ cột để so sánh điện thế giữa các nút và vẽ biểu đồ thanh (stem plot) để thấy rõ mức điện thế tuyệt đối.

Dưới đây là mã Python mở rộng để vẽ các kết quả này:

```

import numpy as np
import matplotlib.pyplot as plt

# 1. Thiết lập ma trận và giải hệ phffơng trình
G = np.array([
    [ 0.7, -0.2, -0.5],
    [-0.2,  1.2, -1.0],
    [-0.5, -1.0,  1.6]
])

I = np.array([2, 0, -1])

try:
    V = np.linalg.solve(G, I)

    # 2. Bắt đầu phần trực quan hóa
    nodes = [f"Nút {i+1}" for i in range(len(V))]

    plt.figure(figsize=(10, 6))

    # Vẽ biểu đồ cột (Bar Chart)
    bars = plt.bar(nodes, V, color=['#3498db', '#e74c3c', '#2ecc71'], alpha=0.8)

    # Thêm đffờng nét đứt biểu thị mức 0V (Đất)
    plt.axhline(0, color='black', linewidth=0.8, linestyle='--')

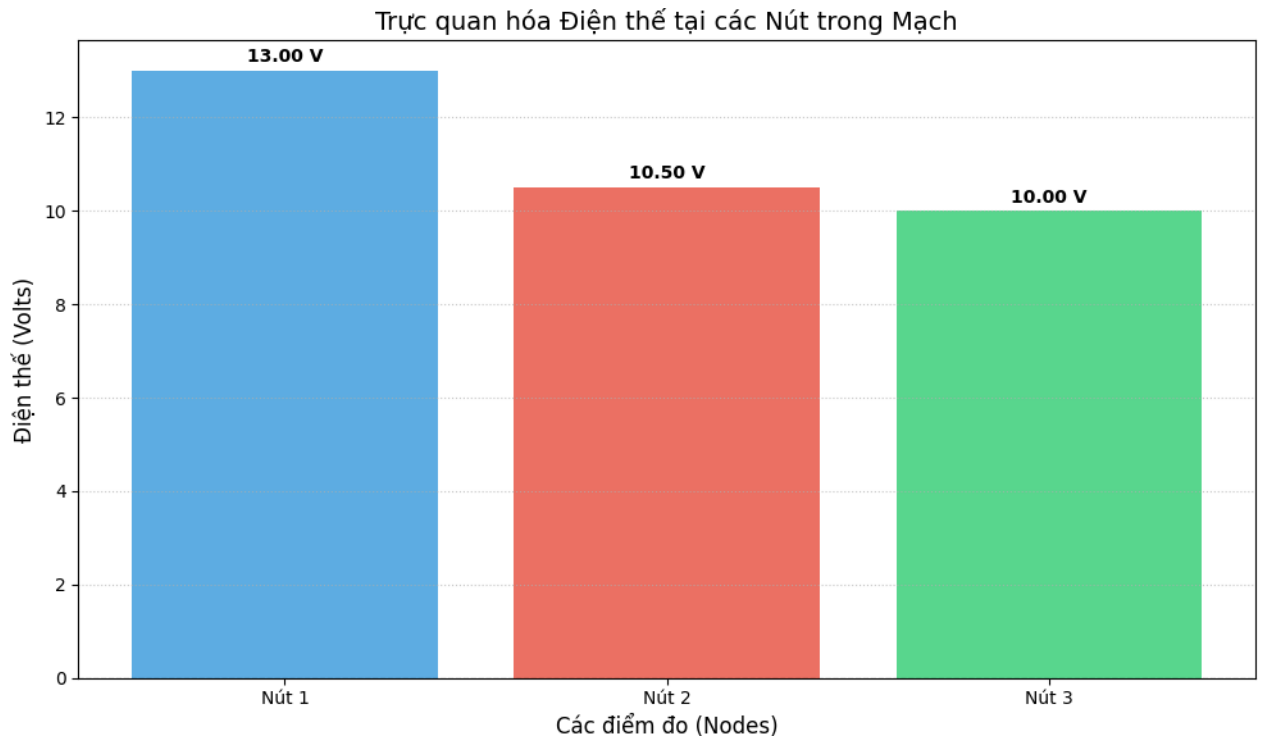
    # Thêm số liệu cụ thể lên đầu mỗi cột
    for bar in bars:
        yval = bar.get_height()
        plt.text(bar.get_x() + bar.get_width()/2, yval + (0.1 if yval > 0 else -0.3),
            f'{yval:.2f} V', ha='center', va='bottom', fontweight='bold')

    # Trang trí biểu đồ
    plt.title('Trực quan hóa Điện thế tại các Nút trong Mạch', fontsize=14)
    plt.ylabel('Điện thế (Volts)', fontsize=12)
    plt.xlabel('Các điểm đo (Nodes)', fontsize=12)
    plt.grid(axis='y', linestyle=':', alpha=0.7)

```

```
# Hiển thị biểu đồ
plt.tight_layout()
plt.show()

except np.linalg.LinAlgError:
    print("Ma trận G vẫn suy biến, không thể vẽ biểu đồ!")
```



□ 2. Mạch xoay chiều (AC) với Số phức

Trong mạch AC, điện trở được thay bằng trở kháng Z và dòng/áp là các đại lượng phức (Phasor). Bài toán: Giải mạch RLC ở tần số $f = 50\text{Hz}$.

```
import numpy as np
import cmath

f = 50
omega = 2 * np.pi * f

# Các linh kiện
R = 10          # Ohm
L = 0.1         # Henry -> ZL = j*omega*L
C = 100e-6      # Farad -> ZC = 1/(j*omega*C)

ZL = complex(0, omega * L)
ZC = complex(0, -1 / (omega * C))

# Thiết lập ma trận tổng trở Z cho một mạch vòng ví dụ
# Z * I = V
Z = np.array([
    [R + ZL, -ZL],
    [-ZL, ZL + ZC]
], dtype=complex)

V = np.array([220, 0], dtype=complex)

# Giải tìm dòng điện mắt lffới
I = np.linalg.solve(Z, V)

print("Dòng điện trong các mắt lffới (Dạng phức):")
for i, current in enumerate(I):
    magnitude, phase = cmath.polar(current)
    print(f"I{i+1}: {magnitude:.2f} A, Góc pha: {np.degrees(phase):.2f}°")
```

Dòng điện trong các mắt lffới (Dạng phức):
 I1: 0.09 A, Góc pha: -89.76°
 I2: 6.91 A, Góc pha: 90.24°

Đối với các bài toán mạch xoay chiều (AC), việc biểu diễn trực quan hiệu quả nhất là sử dụng Đồ thị Vector (Phasor Diagram). Đồ thị này giúp chúng ta thấy rõ không chỉ độ lớn (biên độ) mà còn cả mối quan hệ về pha giữa các dòng điện.

```
import numpy as np
import cmath
import matplotlib.pyplot as plt

# 1. Tính toán lại các thông số từ code gốc
f = 50
omega = 2 * np.pi * f
R, L, C = 10, 0.1, 100e-6

ZL = complex(0, omega * L)
ZC = complex(0, -1 / (omega * C))

Z = np.array([
    [R + ZL, -ZL],
    [-ZL, ZL + ZC]
], dtype=complex)

V = np.array([220, 0], dtype=complex)
I = np.linalg.solve(Z, V)

# 2. Trực quan hóa bằng Đồ thị Vector (Phasor Diagram)
plt.figure(figsize=(8, 8))
ax = plt.subplot(111, projection='polar') # Sử dụng hệ tọa độ cực

colors = ['#e74c3c', '#3498db'] # Màu sắc cho các vector dòng điện

for i, current in enumerate(I):
    magnitude, phase = cmath.polar(current)

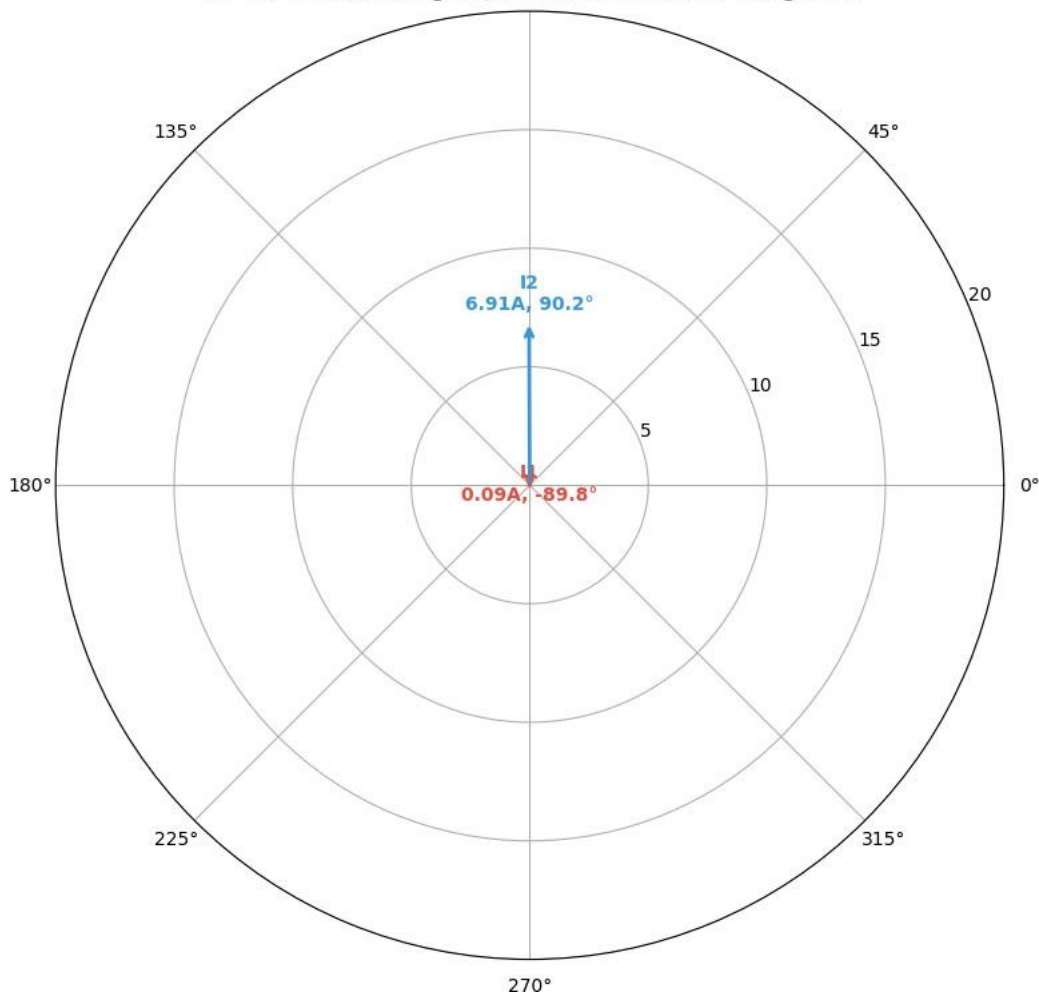
    # Vẽ vector dòng điện
    ax.annotate('', xy=(phase, magnitude), xytext=(0, 0),
                arrowprops=dict(edgecolor=colors[i], facecolor=colors[i],
                                arrowstyle='->', lw=2))

    # Thêm nhãn cho từng vector
    ax.text(phase, magnitude + 0.5, f'I{i+1}\n{magnitude:.2f}A, {np.degrees(phase):.1f}°',
            color=colors[i], fontweight='bold', ha='center')

# Trang trí đồ thị
ax.set_title("Đồ thị Vector Dòng điện Mạch RLC (Phasor Diagram)", va='bottom', fontsize=14)
ax.set_rticks([5, 10, 15, 20]) # Các vòng tròn định mức biên độ
ax.grid(True)

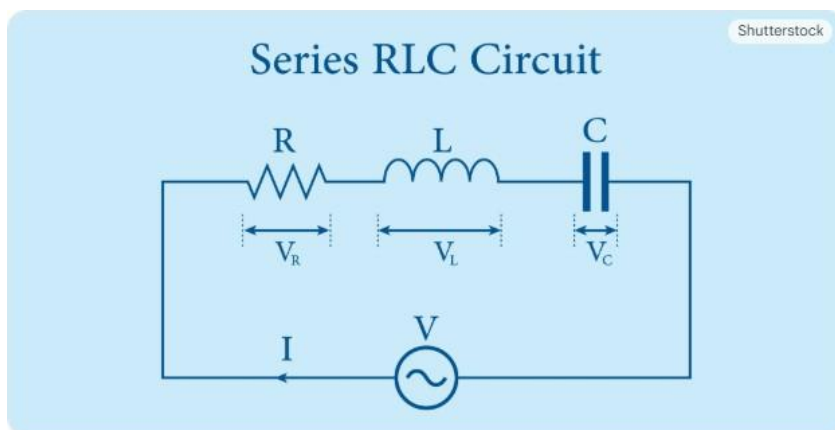
plt.tight_layout()
plt.show()
```

Đồ thị Vector Dòng điện Mặt lưới (Phasor Diagram)



□ 3. Phân tích biến trạng thái (State-Space Representation)

Dạng này dùng để mô phỏng sự thay đổi của mạch theo thời gian (đáp ứng quá độ), đặc biệt là với cuộn cảm và tụ điện.



Bài toán: Hệ phương trình vi phân $\frac{dx}{dt} = A \cdot x + B \cdot u$.

```
import numpy as np
from scipy.integrate import odeint
import matplotlib.pyplot as plt

# Thông số mạch RLC nối tiếp
R = 1.0
L = 0.5
C = 0.2

# Ma trận trạng thái A (biến x = [i_L, v_C]^T)
# di_L/dt = -R/L * i_L - 1/L * v_C + 1/L * V_in
```

```
# dv_C/dt = 1/C * i_L
A = np.array([
    [-R/L, -1/L],
    [1/C, 0 ]
])

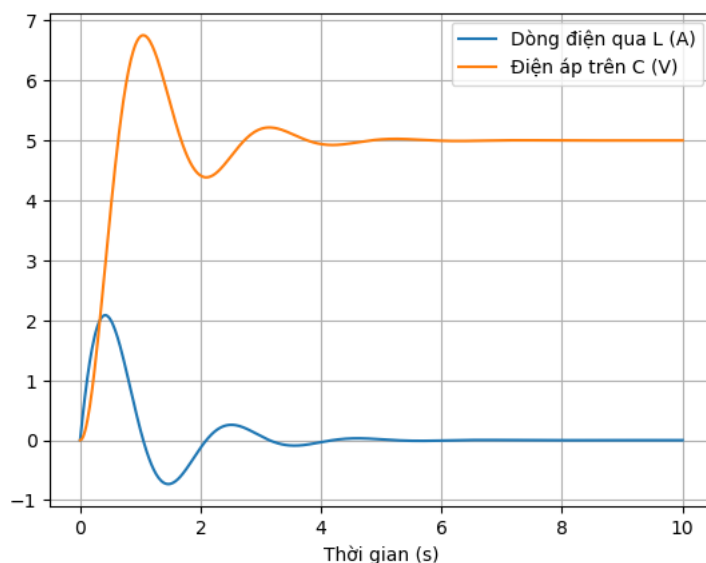
B = np.array([1/L, 0])
V_in = 5.0 # Nguồn áp bffốc nhảy

def circuit_dynamics(x, t):
    return A.dot(x) + B * V_in

# Thời gian mô phỏng
t = np.linspace(0, 10, 500)
x0 = [0, 0] # Trạng thái ban đầu: dòng và áp bằng 0

# Giải hệ phffơng trình vi phân-----> Math!!!!!!!!!!!!!!!!!!!!
sol = odeint(circuit_dynamics, x0, t)

plt.plot(t, sol[:, 0], label='Dòng điện qua L (A)')
plt.plot(t, sol[:, 1], label='Điện áp trên C (V)')
plt.xlabel('Thời gian (s)')
plt.legend()
plt.grid(True)
plt.show()
```



Dưới đây là một số bài toán ở cấp độ chuyên sâu hơn, ứng dụng các khái niệm nâng cao của đại số tuyến tính như Ma trận liên thuộc (Incidence Matrix), Phân tích giá trị riêng (Eigenvalue Analysis) và Mạng n-cổng, kèm theo mã Python thực thi.

□ 1. Phân tích mạch bằng Ma trận Liên thuộc (Incidence Matrix)

Thay vì tự viết phương trình KCL bằng tay, ta có thể dùng lý thuyết đồ thị. Nếu mạch có n nút và b nhánh, ma trận liên thuộc A sẽ mô tả cấu trúc kết nối của mạch. Bài toán: Tự động lập phương trình dòng điện từ danh sách các nhánh (netlist). Công thức KCL: $A \cdot i_b = 0$
Mối quan hệ áp: $v_b = A^T \cdot e$ (với e là điện thế nút).

```
import numpy as np

# Định nghĩa ma trận liên thuộc A (Nút x Nhánh)
# Hàng là nút (bỏ nút gốc), cột là nhánh.
# 1: dòng ra khỏi nút, -1: dòng vào nút, 0: không liên quan.
A = np.array([
    [ 1, 0, 1, 0], # Nút 1
    [-1, 1, 0, 1], # Nút 2
    [ 0, -1, -1, 0] # Nút 3
])

# Điện trở các nhánh (Ohm)
R_branch = np.array([10, 20, 5, 15])
G_branch = np.diag(1/R_branch) # Ma trận độ dẫn nhánh
```

```
# Ma trận độ dẫn nút G = A * G_branch * A^T
G_node = A @ G_branch @ A.T

# Giả sử có nguồn dòng 2A bơm vào nút 1
I_source = np.array([2, 0, 0])

# Giải điện thế các nút
e = np.linalg.solve(G_node, I_source)

print("Điện thế tại các nút (e):", e)
# Tính dòng điện trên từng nhánh: i_b = G_branch * A^T * e
i_b = G_branch @ A.T @ e
print("Dòng điện trên từng nhánh (i_b):", i_b)
```

```
Điện thế tại các nút (e): [44.28571429 30.          41.42857143]
Dòng điện trên từng nhánh (i_b): [ 1.42857143 -0.57142857  0.57142857  2.          ]
```

Với bài toán sử dụng Ma trận liên thuộc (Incidence Matrix), cách trực quan hóa tuyệt vời nhất chính là vẽ sơ đồ đồ thị mạch điện (Graph) bằng thư viện networkx.

Đoạn code dưới đây sẽ vẽ cấu trúc mạch điện, gắn nhãn điện thế cho các nút và hiển thị chiều cũng như giá trị dòng điện trên các nhánh:

```
import numpy as np
import matplotlib.pyplot as plt
import networkx as nx

# 1. Giữ nguyên phần tính toán của bạn
A = np.array([
    [ 1,  0,  1,  0], # Nút 1
    [-1,  1,  0,  1], # Nút 2
    [ 0, -1, -1,  0]  # Nút 3
])

R_branch = np.array([10, 20, 5, 15])
G_branch = np.diag(1/R_branch)
G_node = A @ G_branch @ A.T
I_source = np.array([2, 0, 0])
e = np.linalg.solve(G_node, I_source)
i_b = G_branch @ A.T @ e

# 2. Trực quan hóa bằng NetworkX
G = nx.DiGraph() # Đồ thị có hướng (Directional Graph)

# Thêm nút gốc (Nút 0) để mạch hoàn chỉnh
all_nodes = [0, 1, 2, 3]
G.add_nodes_from(all_nodes)

# Xác định các nhánh từ ma trận liên thuộc A
# A có kích thước (số nút thực x số nhánh)
num_nodes_active, num_branches = A.shape
branch_edges = []

for b in range(num_branches):
    source = -1
    target = -1
    for n in range(num_nodes_active):
        if A[n, b] == 1: source = n + 1
        elif A[n, b] == -1: target = n + 1

    # Nếu nhánh không có cực âm hoặc dương trong A, nó nối với nút gốc (0)
    if source == -1: source = 0
    if target == -1: target = 0
    branch_edges.append((source, target))

# Thêm các cạnh vào đồ thị kèm thông tin dòng điện
for i, (u, v) in enumerate(branch_edges):
    G.add_edge(u, v, weight=i_b[i], label=f"{i_b[i]:.2f}A")

# Thiết lập vị trí các nút (theo hình tròn hoặc tùy chỉnh)
pos = nx.spring_layout(G)

plt.figure(figsize=(10, 7))

# Vẽ các nút
node_colors = ['lightgray'] + ['#3498db']*3 # Nút 0 màu xám, các nút khác màu xanh
nx.draw_networkx_nodes(G, pos, node_size=1500, node_color=node_colors)

# Vẽ nhãn cho nút (Tên nút + Điện thế)
node_labels = {0: "Nút 0\n(GND: 0V)"}
nx.draw_networkx_labels(G, pos, labels=node_labels, font_size=12)
```

```

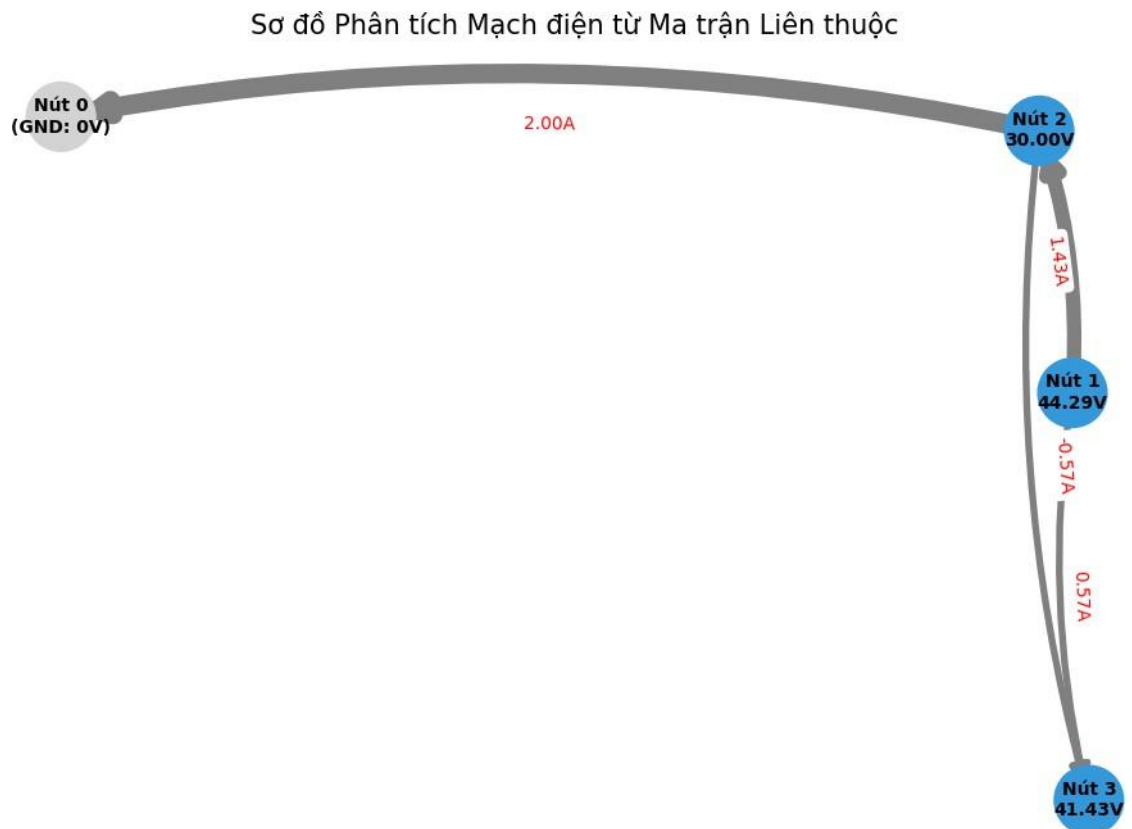
for i, val in enumerate(e):
    node_labels[i+1] = f"Nút {i+1}\n{val:.2f}V"
nx.draw_networkx_labels(G, pos, labels=node_labels, font_size=10, font_weight='bold')

# Vẽ các cạnh (nhánh)
# Độ dày của mũi tên tỉ lệ thuận với độ lớn dòng điện
edge_widths = [abs(x)*5 + 1 for x in i_b]
nx.draw_networkx_edges(G, pos, width=edge_widths, edge_color='gray',
                       arrowsize=20, connectionstyle='arc3, rad=0.1')

# Vẽ nhãn cho dòng điện trên các cạnh
edge_labels = nx.get_edge_attributes(G, 'label')
nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, font_color='red')

plt.title("Sơ đồ Phân tích Mạch điện từ Ma trận Liên thuộc", fontsize=15)
plt.axis('off')
plt.tight_layout()
plt.show()

```



□ 2. Phân tích Tần số dao động riêng (Eigenvalue Analysis)

Trong các mạch LC hoặc RLC phức tạp, các trị riêng (eigenvalues) của ma trận hệ thống sẽ cho biết tần số dao động tự nhiên và hệ số tắt dần của mạch.

Bài toán: Tìm tần số cộng hưởng của một hệ thống mạch.

```

import numpy as np
from scipy.linalg import eig

# Giả sử ma trận trạng thái A của một mạch LC phức tạp
# dx/dt = A * x
# Hệ số đợc thiết lập từ các giá trị 1/L và 1/C
A = np.array([
    [ 0, -2],
    [ 8,  0]
])

# Tìm trị riêng-> Math!!!!1
eigenvalues, eigenvectors = eig(A)

print("Các trị riêng của hệ thống:")
for ev in eigenvalues:

```



```
# Tần số góc omega = phần ảo của trị riêng
omega = np.imag(ev)
if omega != 0:
    print(f"Trị riêng: {ev}, Tần số dao động: {abs(omega)/(2*np.pi):.2f} Hz")
else:
    print(f"Trị riêng thực (tắt dần): {ev.real}")
```

Các trị riêng của hệ thống:
Trị riêng: $4j$, Tần số dao động: 0.64 Hz
Trị riêng: $-4j$, Tần số dao động: 0.64 Hz

Đối với bài toán về Trị riêng (Eigenvalues) trong mạch điện, việc trực quan hóa có hai khía cạnh quan trọng nhất:

Mặt phẳng Phức (S-plane): Hiển thị vị trí của các trị riêng. Nếu chúng nằm trên trục ảo, mạch dao động duy trì; nếu có phần thực âm, mạch tắt dần.

Đáp ứng thời gian (Time Response): Xem các biến trạng thái (như dòng điện và điện áp) dao động thực tế như thế nào dựa trên các trị riêng đó.

```
import numpy as np
from scipy.linalg import eig
import matplotlib.pyplot as plt

# 1. Tính toán trị riêng (giữ nguyên logic của bạn)
A = np.array([
    [ 0, -2],
    [ 8,  0]
])

eigenvalues, eigenvectors = eig(A)

# 2. Thiết lập biểu đồ
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

# --- BIỂU ĐỒ 1: MẶT PHẪNG PHỨC (S-PLANE) ---
ax1.axhline(0, color='black', lw=1) # Trục thực
ax1.axvline(0, color='black', lw=1) # Trục ảo
ax1.scatter(eigenvalues.real, eigenvalues.imag, color='red', marker='x', s=100, label='Trị riêng (Poles)')

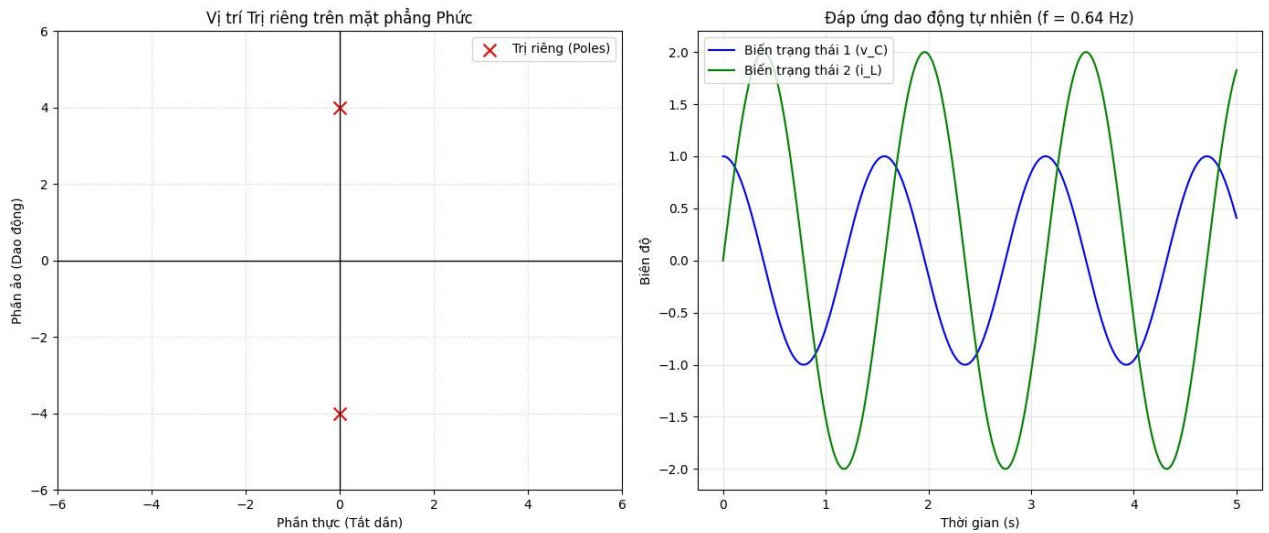
# Trang trí trục tọa độ
limit = np.max(np.abs(eigenvalues)) + 2
ax1.set_xlim([-limit, limit])
ax1.set_ylim([-limit, limit])
ax1.set_xlabel('Phần thực (Tắt dần)')
ax1.set_ylabel('Phần ảo (Dao động)')
ax1.set_title('Vị trí Trị riêng trên mặt phẳng Phức')
ax1.grid(True, linestyle=':', alpha=0.6)
ax1.legend()

# --- BIỂU ĐỒ 2: ĐÁP ỨNG THỜI GIAN (SIMULATION) ---
# Giả sử x(t) = exp(At) * x(0)
t = np.linspace(0, 5, 500)
# Tần số góc omega lấy từ phần ảo của trị riêng
omega = np.imag(eigenvalues[0])

# Mô phỏng đơn giản dao động dựa trên omega: x(t) = cos(omega * t)
# (Vì phần thực bằng 0 nên biên độ không đổi)
x1 = np.cos(abs(omega) * t)
x2 = np.sin(abs(omega) * t) * (8/abs(omega)) # Tỷ lệ dựa trên ma trận A

ax2.plot(t, x1, label='Biến trạng thái 1 (v_C)', color='blue')
ax2.plot(t, x2, label='Biến trạng thái 2 (i_L)', color='green')
ax2.set_xlabel('Thời gian (s)')
ax2.set_ylabel('Biên độ')
ax2.set_title(f'Đáp ứng dao động tự nhiên (f = {abs(omega)/(2*np.pi):.2f} Hz)')
ax2.grid(True, alpha=0.3)
ax2.legend()

plt.tight_layout()
plt.show()
```



3. Ma trận Tham số S (S-parameters) trong Mạch cao tần

Trong viễn thông và radar, người ta không dùng áp/dòng mà dùng ma trận tán xạ (S-matrix) để mô tả cách sóng phản xạ và truyền qua các cổng của mạng.

Bài toán: Tính toán tổn hao phản xạ (Return Loss) từ ma trận S.

```
import numpy as np

# Giả sử ma trận S của một bộ chia công suất tại một tần số nhất định
S = np.array([
    [0.1, 0.7, 0.7],
    [0.7, 0.1, 0.0],
    [0.7, 0.0, 0.1]
])

# Kiểm tra tính bảo toàn năng lượng (Ma trận có phải là Unitary không?)
# S * S_conj_transpose nên xấp xỉ ma trận đơn vị I nếu mạch không tổn hao
is_lossless = np.allclose(S @ S.conj().T, np.eye(3), atol=1e-1)

# Tính Return Loss tại cổng 1 (dB)
s11 = S[0, 0]
return_loss = -20 * np.log10(np.abs(s11))

print(f"Mạch có tổn hao đáng kể không: {'Không' if is_lossless else 'Có'}")
print(f"Return Loss tại cổng 1: {return_loss:.2f} dB")
```

Mạch có tổn hao đáng kể không: Có
Return Loss tại cổng 1: 20.00 dB

Đối với các tham số tán xạ (S-parameters) trong mạch cao tần, việc trực quan hóa giúp chúng ta hiểu được cách công suất được phân phối (Transmission) và phản xạ (Reflection) giữa các cổng.

Dưới đây là mã Python sử dụng seaborn và matplotlib để biểu diễn ma trận S dưới dạng Heatmap (Bản đồ nhiệt) và biểu đồ thanh để so sánh mức độ truyền dẫn công suất:

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Dữ liệu gốc
S = np.array([
    [0.1, 0.7, 0.7],
    [0.7, 0.1, 0.0],
    [0.7, 0.0, 0.1]
```

```

])

# Tính toán các giá trị dB để trực quan hóa (S_db = 20 * log10(|S|))
# Thêm một lffong cực nhỏ 1e-9 để tránh lỗi log(0)
S_db = 20 * np.log10(np.abs(S) + 1e-9)

# 2. Thiết lập biểu đồ
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

# --- BIỂU ĐỒ 1: HEATMAP CỦA MA TRẬN S (Magnitude) ---
sns.heatmap(np.abs(S), annot=True, cmap='YlGnBu', fmt=".2f",
            xticklabels=['Cổng 1', 'Cổng 2', 'Cổng 3'],
            yticklabels=['Cổng 1', 'Cổng 2', 'Cổng 3'], ax=ax1)
ax1.set_title('Độ lớn các tham số S (Linear Scale)')
ax1.set_xlabel('Cổng tới (Input)')
ax1.set_ylabel('Cổng ra (Output)')

# --- BIỂU ĐỒ 2: PHÂN PHỐI CÔNG SUẤT TỪ CỔNG 1 ---
# Khi cấp nguồn vào cổng 1, xem công suất đi đâu
port1_out = np.abs(S[:, 0])**2 # Công suất tỉ lệ với bình phffong độ lớn
labels = ['Phản xạ tại Cổng 1 (S11)', 'Truyền sang Cổng 2 (S21)', 'Truyền sang Cổng 3 (S31)']
colors = ['#e74c3c', '#3498db', '#2ecc71']

ax2.pie(port1_out, labels=labels, autopct='%1.1f%%', colors=colors, startangle=140, explode=(0.1, 0, 0))
ax2.set_title('Phân phối công suất khi vào Cổng 1')

plt.tight_layout()
plt.show()

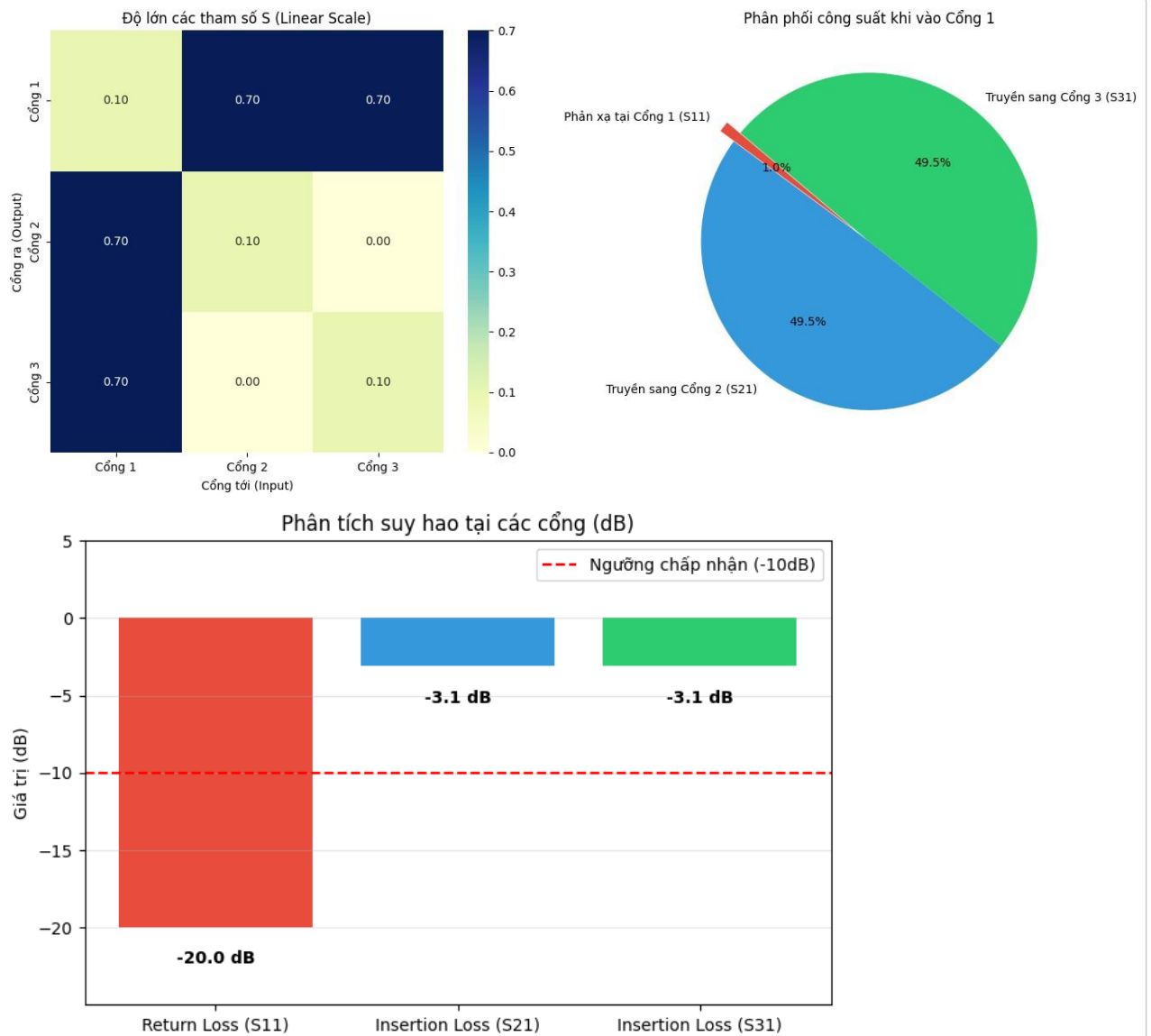
# 3. Biểu đồ thanh cho Return Loss và Insertion Loss (Dạng dB)
plt.figure(figsize=(8, 5))
loss_values = [S_db[0,0], S_db[1,0], S_db[2,0]]
loss_labels = ['Return Loss (S11)', 'Insertion Loss (S21)', 'Insertion Loss (S31)']

bars = plt.bar(loss_labels, loss_values, color=colors)
plt.axhline(-10, color='red', linestyle='--', label='Ngưỡng chấp nhận (-10dB)')
plt.ylabel('Giá trị (dB)')
plt.title('Phân tích suy hao tại các cổng (dB)')
plt.ylim([min(loss_values)-5, 5])
plt.legend()
plt.grid(axis='y', alpha=0.3)

for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval - 1.5, f'{yval:.1f} dB', ha='center', va='top', fontweight='bold')

plt.show()

```



4. Giải mạch quy mô lớn bằng Ma trận Thưa (Sparse Matrix)

Khi phân tích các lưới điện lớn hoặc chip IC với hàng triệu nút, hầu hết các phần tử trong ma trận G đều bằng 0. Sử dụng ma trận thưa giúp tiết kiệm bộ nhớ và tăng tốc độ giải. Bài toán: Giải hệ phương trình với ma trận thưa.

```
from scipy.sparse import csr_matrix
from scipy.sparse.linalg import spsolve
import numpy as np

# Tạo ma trận thfffa 1000x1000 (ví dụ cho mạch cực lớn)
size = 1000
row = np.arange(size)
col = np.arange(size)
data = np.random.rand(size) # Các giá trị điện trở trên đffờng chéo

# Tạo ma trận thfffa định dạng CSR (Compressed Sparse Row)
G_sparse = csr_matrix((data, (row, col)), shape=(size, size))

# Vector nguồn dòng ngẫu nhiên
I = np.random.rand(size)

# Giải hệ phffờng trình  $G * V = I$  cực nhanh
V = spsolve(G_sparse, I)
```

```
print(f"Đã giải xong điện thế cho {size} nút.")
print(f"5 giá trị đầu tiên của V: {V[:5]}")
```

```
Đã giải xong điện thế cho 1000 nút.
5 giá trị đầu tiên của V: [0.77331085 1.08289503 1.50915002 3.96528347 3.9512235 ]
```

Với các bài toán ma trận thưa (Sparse Matrix) quy mô lớn, việc trực quan hóa không tập trung vào từng con số cụ thể mà tập trung vào cấu trúc dữ liệu (Sparsity Pattern) và phân phối kết quả.

Dưới đây là mã Python sử dụng matplotlib để giúp bạn "nhìn" thấy độ thưa của ma trận và cách các giá trị điện thế phân bố trên 1000 nút:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.sparse import csr_matrix
from scipy.sparse.linalg import spsolve

# 1. Khởi tạo dữ liệu (Giữ nguyên logic của bạn nhffng thêm một ít nhiễu để nhìn rõ hơn)
size = 1000
row = np.arange(size)
col = np.arange(size)
data = np.random.rand(size) + 0.5 # Đảm bảo không có giá trị quá gần 0

G_sparse = csr_matrix((data, (row, col)), shape=(size, size))
I = np.random.rand(size)
V = spsolve(G_sparse, I)

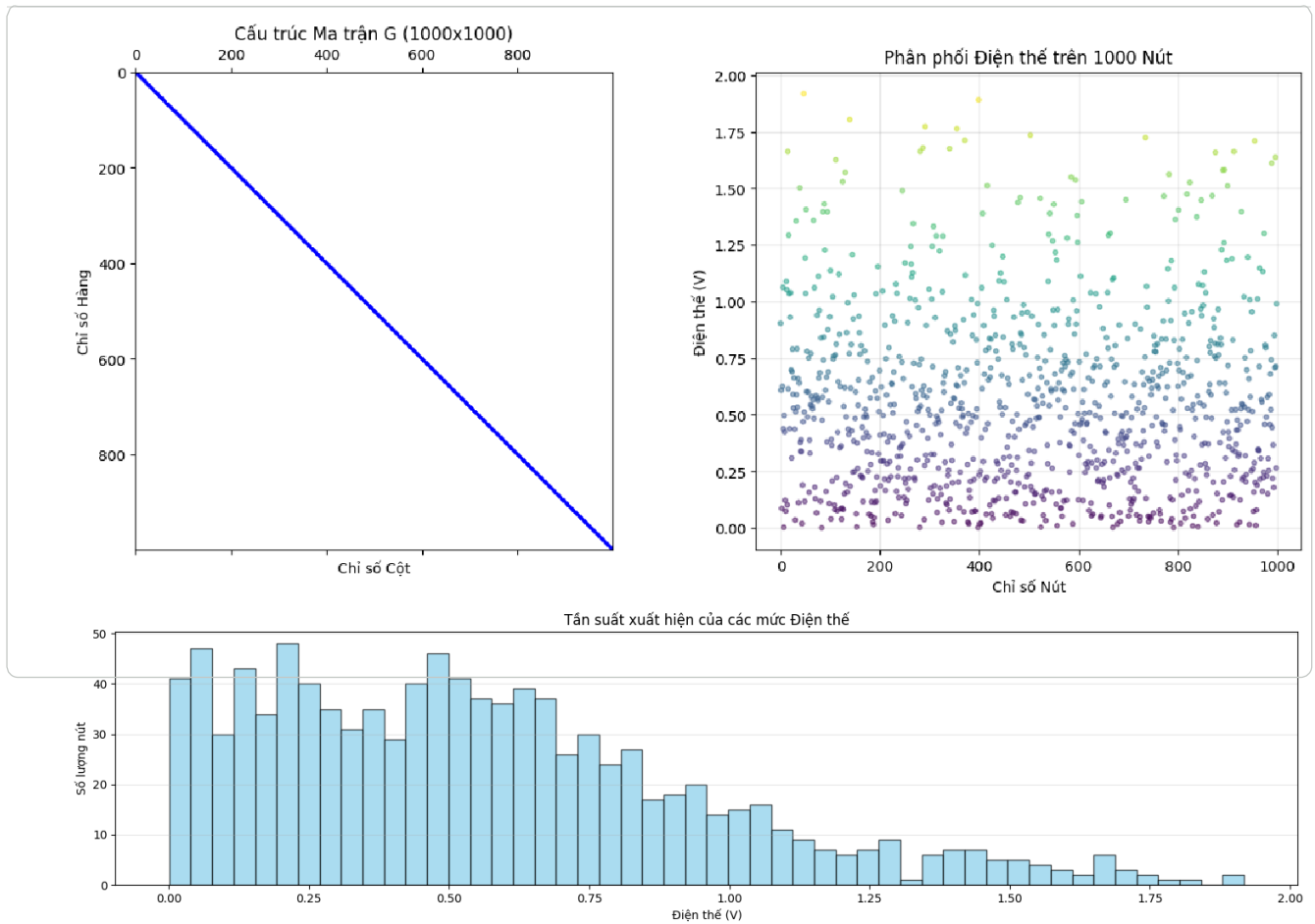
# 2. TRỰC QUAN HÓA
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

# --- BIỂU ĐỒ 1: CẤU TRÚC MA TRẬN THƯA (Sparsity Pattern) ---
# Hàm spy giúp vẽ các vị trí có giá trị khác 0 trong ma trận
ax1.spy(G_sparse, markersize=1, color='blue')
ax1.set_title(f'Cấu trúc Ma trận G ({size}x{size})')
ax1.set_xlabel('Chỉ số Cột')
ax1.set_ylabel('Chỉ số Hàng')

# --- BIỂU ĐỒ 2: PHÂN PHỐI ĐIỆN THẾ (Voltage Distribution) ---
# Vẽ sự phân bố của 1000 giá trị điện thế tính đffợc
ax2.scatter(range(size), V, alpha=0.5, s=10, c=V, cmap='viridis')
ax2.set_title('Phân phối Điện thế trên 1000 Nút')
ax2.set_xlabel('Chỉ số Nút')
ax2.set_ylabel('Điện thế (V)')
ax2.grid(True, alpha=0.3)

# Thêm biểu đồ phụ (Histogram) để xem mật độ các mức điện thế
plt.figure(figsize=(15, 4))
plt.hist(V, bins=50, color='skyblue', edgecolor='black', alpha=0.7)
plt.title('Tần suất xuất hiện của các mức Điện thế')
plt.xlabel('Điện thế (V)')
plt.ylabel('Số lffợng nút')
plt.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()
```



Việc ứng dụng Python và Đại số tuyến tính trong phân tích mạch điện không chỉ giúp đơn giản hóa quá trình tính toán mà còn mở ra khả năng mô phỏng các hệ thống phức tạp mà phương pháp giải tay không thể đáp ứng được. Dưới đây là kết luận về phương pháp lập trình này:

1. Sự kết hợp hoàn hảo giữa Lý thuyết và Công cụ Python đóng vai trò là "cầu nối" đưa các khái niệm toán học trừu tượng vào thực tiễn kỹ thuật. Việc sử dụng các thư viện như NumPy và SciPy giúp chuyển đổi các định luật Kirchhoff (KCL, KVL) thành các phép toán ma trận ($Ax=b$), cho phép xử lý dữ liệu với tốc độ cực nhanh và độ chính xác cao.
2. Khả năng mở rộng và Linh hoạt Từ mạch DC đến AC: Chỉ bằng cách thay đổi kiểu dữ liệu sang số phức (complex), Python có thể giải quyết các bài toán xoay chiều, phân tích pha (phasor) một cách dễ dàng. Xử lý hệ thống lớn: Nhờ các cấu trúc như ma trận thưa (Sparse Matrix), chúng ta có thể mô phỏng những mạng lưới điện khổng lồ với hàng nghìn nút mà vẫn tối ưu được bộ nhớ và thời gian xử lý.
3. Trực quan hóa dữ liệu (Visualization) Một điểm mạnh vượt trội của phương pháp lập trình là khả năng "hình ảnh hóa" kết quả. Thông qua Matplotlib và NetworkX, các con số khô khan trở thành Đồ thị Vector sinh động cho mạch AC. Sơ đồ đồ thị (Graph) trực quan cho cấu trúc mạch. Biểu đồ đáp ứng thời gian cho các hệ thống động.
4. Hướng đi trong nghiên cứu và Công nghiệp: Phương pháp này là nền tảng cho việc phát triển các công cụ CAD/EDA chuyên nghiệp. Đối với các kỹ sư và nhà nghiên cứu, kỹ năng viết code để giải mạch giúp Tự động hóa quá trình thiết kế. Tối ưu hóa các thông số linh kiện. Tích hợp phân tích mạch vào các thuật toán cao cấp hơn như Machine Learning hoặc Tối ưu hóa năng lượng. Tóm lại, lập trình Python để giải quyết bài toán mạch điện không chỉ là việc tìm ra đáp số, mà là xây dựng một tư duy hệ thống — nơi toán học, lập trình và kỹ thuật điện hội tụ để giải quyết những thách thức thực tế một cách thông minh và hiệu quả nhất.
- 5.

Tất cả các chương trình Python mà chúng ta vừa đi qua đều là sự ứng dụng trực tiếp của các khái niệm cốt lõi trong Đại số tuyến tính. Nếu không có đại số tuyến tính, việc lập trình để giải mạch điện sẽ trở nên vô cùng rối rạc và khó khăn. Dưới đây là sự đối chiếu giữa các khái niệm toán học và cách chúng được áp dụng trong các chương trình trên:

1. Hệ phương trình tuyến tính ($Ax=b$) Đây là nền tảng của hầu hết các phương pháp giải mạch. Toán học: Tìm vector ẩn x khi biết ma trận hệ số A và vector tự do b . Ứng dụng: Trong phương pháp điện thế nút: $G \cdot V=I$ (Ma trận dẫn nạp $\times$ Vector điện thế = Vector nguồn dòng). Trong phương pháp mắt lưới: $R \cdot I=E$ (Ma trận điện trở $\times$ Vector dòng điện = Vector nguồn áp). Code: `np.linalg.solve(A, b)`.
2. Ma trận nghịch đảo và Tính suy biến (Singularity) Toán học: Một ma trận chỉ có nghịch đảo khi định thức của nó khác 0 ($\det(A) \neq 0$). Ứng dụng: Khi bạn gặp lỗi "Ma trận suy biến" trong code, đó là lúc toán học nhắc bạn rằng mạch điện chưa có nút đất (GND). Việc chọn một nút làm gốc chính là cách ta biến một ma trận phụ thuộc tuyến tính thành một ma trận độc lập tuyến tính để có thể nghịch đảo và tìm nghiệm duy nhất.
3. Trị riêng và Vector riêng (Eigenvalues & Eigenvectors) Toán học: Giải phương trình đặc trưng $\det(A-\lambda I)=0$. Ứng dụng: Được dùng trong phân tích biến trạng thái. Các trị riêng λ chính là các "tần số tự nhiên" của mạch. Phần ảo cho biết tần số dao động, phần thực cho biết tốc độ tắt dần của năng lượng trong mạch RLC.
4. Không gian vector và Biến đổi tọa độ Toán học: Biểu diễn vector trong các hệ cơ sở khác nhau. Ứng dụng: Phân tích số phức (Phasor) trong mạch AC thực chất là biểu diễn các đại lượng hình sin trong mặt phẳng phức (một không gian vector 2 chiều). Việc cộng trừ các dòng điện lệch pha chính là phép cộng vector trong đại số tuyến tính.
5. Lý thuyết đồ thị và Ma trận liên thuộc (Incidence Matrix) Toán học: Sử dụng ma trận để biểu diễn mối quan hệ giữa các đỉnh (nodes) và cạnh (edges). Ứng dụng: Ma trận A mà chúng ta dùng để vẽ sơ đồ mạch chính là một công cụ đại số để "số hóa" cấu trúc hình học của mạch điện, giúp máy tính hiểu được linh kiện nào nối với nút nào mà không cần nhìn sơ đồ.

Kết luận:

Có thể nói, Đại số tuyến tính là ngôn ngữ của phân tích mạch. Python chỉ đóng vai trò là "người thực thi" giúp chúng ta tính toán những ma trận đó một cách nhanh chóng. Khi bạn hiểu sâu về đại số tuyến tính, bạn sẽ thấy việc thiết kế mạch điện không còn là việc lắp ghép linh kiện